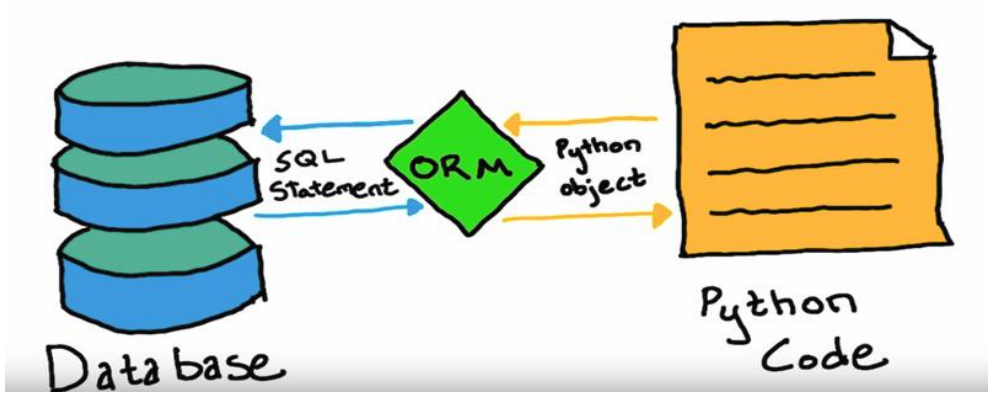
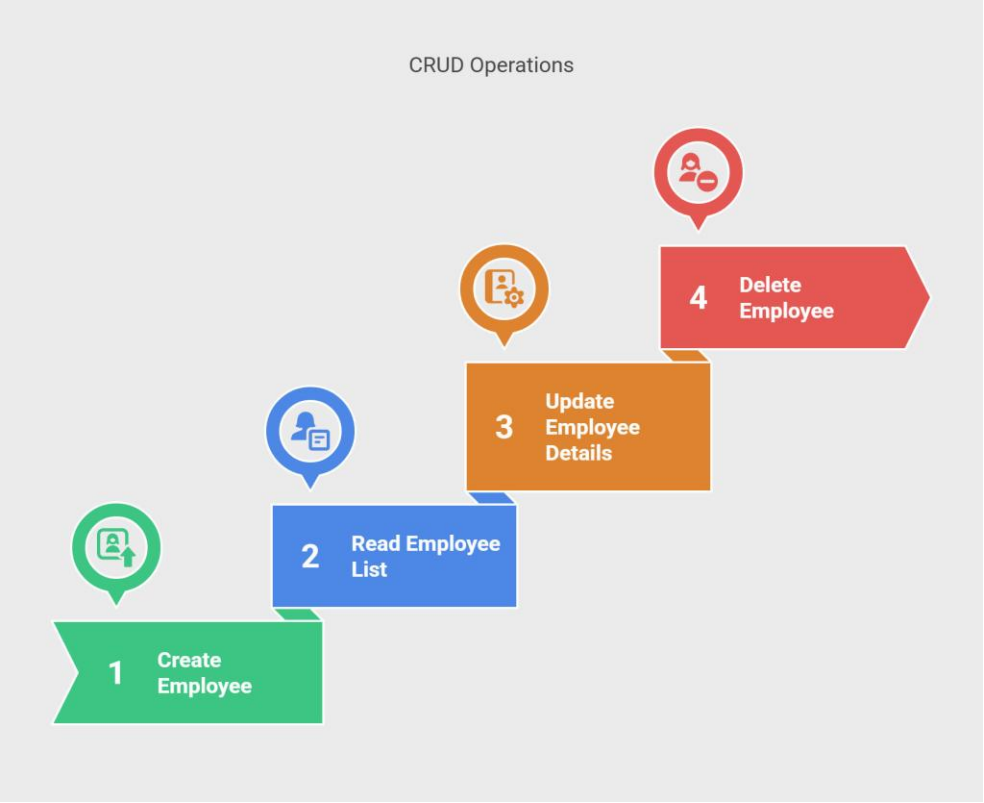
# The Database Bridge (SQLAlchemy & ORMs)

- **The Problem**: Writing raw SQL strings in Python (e.g., "SELECT * FROM crops WHERE yield > 100") is prone to errors, hard to debug, and vulnerable to security risks like SQL injection.
- **The Solution**: **Object-Relational Mapping (ORM)**.
- **Concept**: An ORM like SQLAlchemy maps database tables to Python **Classes**. Instead of writing SQL, you interact with standard Python objects.
- **Benefit**: Your code becomes "Database Agnostic" – you can switch from SQLite to PostgreSQL or MySQL without changing your core logic.



# The Lifecycle of Data (CRUD Operations)

- **Definition**: CRUD represents the four essential functions of any persistent storage system:
  - **Create**: Adding new data (e.g., registering a new farm in the database).
  - **Read**: Retrieving information (e.g., querying last year's rainfall stats).
  - **Update**: Modifying existing records (e.g., updating a patient's health status).
  - **Delete**: Removing records (e.g., clearing temporary test data).
- **Workflow**: We use a "Session" to manage these transactions, ensuring that either the whole operation succeeds or none of it does (Atomicity).



# NoSQL – Handling the Unstructured

- **Beyond Tables**: SQL is "Relational" and uses strict schemas. **NoSQL** (Not Only SQL) is designed for data that doesn't fit neatly into rows and columns.
- **Document Stores (MongoDB)**: Instead of tables, data is stored in JSON-like documents.
- **Use Case**: Perfect for social media feeds, sensor logs with varying fields, or real-time application data where the structure changes frequently.